

# When personalization deters delegation to algorithms — Replication analysis

Lara Suraci & Irenaeus Wolff

26 Juni 2026

## Contents

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Data import and preparation</b>                                         | <b>3</b>  |
| 1.1      | Shared plotting constants . . . . .                                        | 3         |
| 1.2      | Load raw data . . . . .                                                    | 3         |
| 1.3      | Preregistered exclusions . . . . .                                         | 3         |
| 1.4      | Build participant-level Part-1 preference measures . . . . .               | 4         |
| <b>2</b> | <b>Descriptive overview</b>                                                | <b>6</b>  |
| 2.1      | Choice quality and payoffs . . . . .                                       | 6         |
| 2.2      | Round-level delegation profile . . . . .                                   | 6         |
| 2.3      | Figure 1 — Average delegation over decisions . . . . .                     | 7         |
| <b>3</b> | <b>Primary test 1 — First-contact delegation (round 1)</b>                 | <b>9</b>  |
| <b>4</b> | <b>Primary test 2 — Delegation across repeated decisions</b>               | <b>10</b> |
| 4.1      | Participant-level Wilcoxon–Mann–Whitney test . . . . .                     | 10        |
| 4.2      | Figure 3 — ECDF of participant-level average delegation . . . . .          | 10        |
| 4.3      | Preregistered regression specification . . . . .                           | 11        |
| 4.4      | Supporting regressions . . . . .                                           | 12        |
| <b>5</b> | <b>Categorical non-delegation</b>                                          | <b>14</b> |
| 5.1      | Comparisons of never-delegators vs. delegators . . . . .                   | 15        |
| <b>6</b> | <b>Responses to feedback (win-stay / lose-shift)</b>                       | <b>18</b> |
| <b>7</b> | <b>Exploratory mechanisms</b>                                              | <b>20</b> |
| 7.1      | Coefficient model (interaction with performance expectations) . . . . .    | 20        |
| 7.2      | Figure A.1 — Condition-specific marginal effects . . . . .                 | 21        |
| 7.3      | Figure A.2 — Expected non-preferred and completely weird choices . . . . . | 23        |
| 7.4      | Treatment comparison of expected algorithmic errors . . . . .              | 23        |
| <b>8</b> | <b>Post-task reasons</b>                                                   | <b>26</b> |
| 8.1      | Figure 4 — Reasons by treatment and delegation type . . . . .              | 26        |
| 8.2      | Aggregate reason patterns . . . . .                                        | 27        |
| 8.3      | LPM: never-delegation on reasons . . . . .                                 | 28        |
| <b>9</b> | <b>Session information</b>                                                 | <b>29</b> |

```
library(dplyr)
library(tidyr)
library(ggplot2)
```

```
library(scales)
library(Exact)      # Boschloo exact test
library(fixest)     # LPM with fixed effects + clustered SE
library(lmtest)     # robust coefficient tests
library(sandwich)   # HC2 standard errors
library(texreg)     # regression tables
```

# 1 Data import and preparation

## 1.1 Shared plotting constants

To keep figures visually consistent we fix the treatment colour palette once.

```
# Treatment colours used throughout (Pref = preference-based, EV = optimisation)
col_pref <- "#4E79A7"
col_ev   <- "#D55E00"
algo_cols <- c("Pref" = col_pref, "EV" = col_ev)
```

## 1.2 Load raw data

Paths are relative to this script's location (replication-package layout).

```
choices <- read.delim("260206_1528_537_lotteryChoices.xls")
subjects <- read.csv("allSubjects.csv", header = TRUE)
prolificData <- read.csv("prolific_demographic_export_approved.csv")

choicesP1 <- subset(choices, Part == 1)
```

## 1.3 Preregistered exclusions

We apply the preregistered exclusion rules: failed bot/slider check at first attempt, failed comprehension checks after the second attempt, failure to confirm careful participation (`Reliable < 6`), disclosed AI usage, incomplete data (`Delegated < 0`), and the log-time “too fast” rule.

```
# "Too fast" flag based on the preregistered log-completion-time rule
prolificData$logComplTime <- log(prolificData$Time.taken)
prolificData$tooFast <- 1 * (
  prolificData$logComplTime <
    mean(log(prolificData$Time.taken)) - 2 * sd(log(prolificData$Time.taken))
)

keep_ids <- prolificData$Participant.id[prolificData$tooFast == 0]

choices <- subset(
  choices,
  BotCheck1attempt == 1 &
    Part1CQsPassedOnAttempt <= 2 &
    Part2CQsPassedOnAttempt <= 2 &
    Reliable >= 6 &
    AIUsageInStudy == 0 &
    Delegated >= 0 &
    ProlificID %in% keep_ids
)

subjects <- subset(subjects, ProlificID %in% choices$ProlificID)
choicesP1 <- subset(choicesP1, ProlificID %in% choices$ProlificID)

# Final analysis sample (paper: N = 233; EV = 118, Pref = 115)
cat("Final sample size per condition (1 = EV, 0 = Preference-based):\n")

## Final sample size per condition (1 = EV, 0 = Preference-based):
```

```
table(subjects$EVALgorithm)
```

```
##
##    0    1
## 115 118
```

```
# Convenience label for the algorithm condition
choices <- choices %>%
  mutate(Algorithm = ifelse(EVALgorithm == 1, "EV", "Pref"))
```

## 1.4 Build participant-level Part-1 preference measures

These variables (revealed risk preferences from Part 1, per-task confidence, and post-task expectations) are merged onto the decision-level `choices` data for use in the regressions and the appendix coefficient plot.

```
# EV-maximising choice in each Part-1 task
choicesP1 <- choicesP1 %>%
  mutate(EVChoice = case_when(
    grepl("^P", ChoiceType) & PBetChosen == 1 ~ 1,
    grepl("^D", ChoiceType) & PBetChosen == 0 ~ 1,
    TRUE ~ 0
  ))

# "True" P-bet choice (less-risky choice when it was NOT EV-maximising)
choicesP1 <- choicesP1 %>%
  mutate(truePChoice = case_when(
    grepl("^D", ChoiceType) & PBetChosen == 1 ~ 1,
    TRUE ~ 0
  ))

# Participant-level Part-1 summaries
p1_summaries <- choicesP1 %>%
  group_by(ProlificID) %>%
  summarise(
    EVChoices      = sum(EVChoice),
    truePChoices   = sum(truePChoice),
    PBet_count     = sum(PBetChosen),
    .groups = "drop"
  )

choices <- choices %>%
  left_join(p1_summaries, by = "ProlificID") %>%
  # per-task Part-1 confidence (matched on participant x option set)
  left_join(
    choicesP1 %>% select(ProlificID, OptionsetID, ConfidenceP1 = Confidence),
    by = c("ProlificID", "OptionsetID")
  ) %>%
  # participant-level demographics and post-task expectations
  left_join(
    subjects %>% select(
      ProlificID, Female, Age, Education, HHIncome,
      ProbOfBadComputerChoices, ProbOfAbsurdComputerChoices, ComputerRiskTaking
    ),
    by = "ProlificID"
```



## 2 Descriptive overview

### 2.1 Choice quality and payoffs

Choice quality among non-delegated decisions and the payoff advantage of delegation (paper, Section “Primary test: first-contact delegation”).

```
# Share of non-dominated self-chosen options, by condition
choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm) %>%
  summarise(
    avNonDom = mean(Nondominated[Delegated == 0 & Option > 0], na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 2
##   Algorithm avNonDom
##   <chr>      <dbl>
## 1 EV        0.341
## 2 Pref      0.351
```

```
# Average payoff: delegated vs. non-delegated decisions
# (paper: 243.49 points delegated vs. 195.18 points non-delegated, ~25% higher)
choices %>%
  filter(Part == 2) %>%
  group_by(Delegated) %>%
  summarise(meanPayoff = mean(RelevantPayoff, na.rm = TRUE), .groups = "drop")
```

```
## # A tibble: 2 x 2
##   Delegated meanPayoff
##   <int>      <dbl>
## 1      0      195.
## 2      1      243.
```

### 2.2 Round-level delegation profile

```
choicesAv <- choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm, WithinPartDecisionNumber) %>%
  summarise(
    avDeleg = mean(Delegated, na.rm = TRUE),
    avNonDom = mean(Nondominated[Delegated == 0 & Option > 0], na.rm = TRUE),
    .groups = "drop"
  )
choicesAv
```

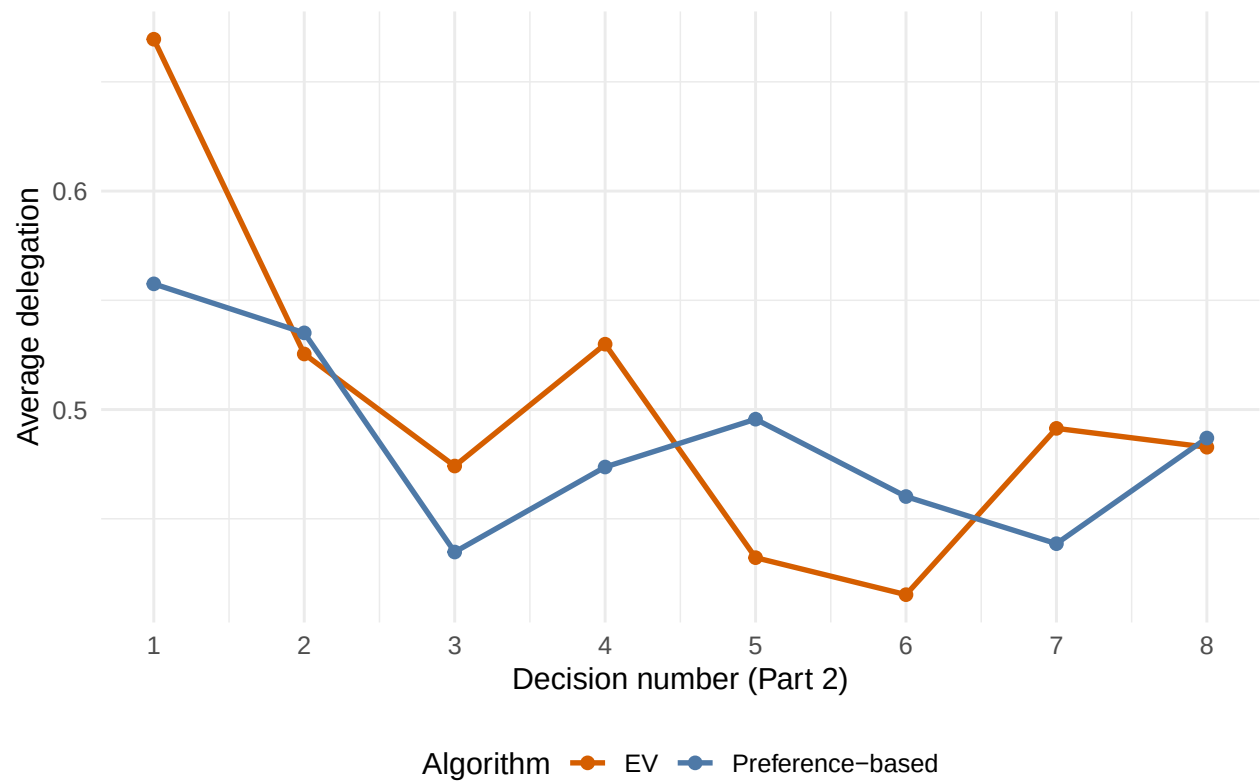
```
## # A tibble: 16 x 4
##   Algorithm WithinPartDecisionNumber avDeleg avNonDom
##   <chr>                <int>    <dbl>    <dbl>
## 1 EV                    1    0.669    0.342
## 2 EV                    2    0.525    0.214
## 3 EV                    3    0.474     0.3
## 4 EV                    4    0.530    0.333
## 5 EV                    5    0.432    0.269
```

|            |   |       |       |
|------------|---|-------|-------|
| ## 6 EV    | 6 | 0.415 | 0.464 |
| ## 7 EV    | 7 | 0.491 | 0.356 |
| ## 8 EV    | 8 | 0.483 | 0.433 |
| ## 9 Pref  | 1 | 0.558 | 0.4   |
| ## 10 Pref | 2 | 0.535 | 0.327 |
| ## 11 Pref | 3 | 0.435 | 0.344 |
| ## 12 Pref | 4 | 0.474 | 0.267 |
| ## 13 Pref | 5 | 0.496 | 0.429 |
| ## 14 Pref | 6 | 0.460 | 0.333 |
| ## 15 Pref | 7 | 0.439 | 0.391 |
| ## 16 Pref | 8 | 0.487 | 0.322 |

## 2.3 Figure 1 — Average delegation over decisions

```
fig1 <- choicesAv %>%
  mutate(Algorithm = factor(Algorithm, levels = c("EV", "Pref"),
                             labels = c("EV", "Preference-based"))) %>%
  ggplot(aes(x = WithinPartDecisionNumber, y = avDeleg,
             color = Algorithm, group = Algorithm)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 1:8) +
  scale_color_manual(values = c("EV" = col_ev, "Preference-based" = col_pref)) +
  labs(x = "Decision number (Part 2)",
       y = "Average delegation",
       color = "Algorithm") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

fig1
```



```
ggsave("Figure1_AverageDelegation.pdf", fig1, width = 7, height = 4.5)
```

### 3 Primary test 1 — First-contact delegation (round 1)

Preregistered one-sided Boschloo exact test on round-1 delegation (paper: 66.9% vs. 55.8%,  $p = 0.0413$ ).

```
tab <- table(
  Delegated = choices$Delegated[choices$Part == 2 & choices$WithinPartDecisionNumber ==
    ↪ 1],
  EVALgorithm = choices$EVALgorithm[choices$Part == 2 & choices$WithinPartDecisionNumber
    ↪ == 1]
)

# Delegation share by condition (column 2 = EV)
prop.table(tab, margin = 2)

##           EVALgorithm
## Delegated      0      1
##      0 0.4424779 0.3305085
##      1 0.5575221 0.6694915

# One-sided Boschloo exact test (H1: EV > Pref)
exact.test(tab, method = "Boschloo", alternative = "greater", to.plot = FALSE)$p.value

## [1] 0.0412565
```

## 4 Primary test 2 — Delegation across repeated decisions

### 4.1 Participant-level Wilcoxon–Mann–Whitney test

Preregistered one-sided WMW test on participant-average delegation (paper:  $W = 6428$ ,  $p = 0.2425$ ).

```
delegAv <- choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm, ProlificID) %>%
  summarise(avDeleg = mean(Delegated, na.rm = TRUE), .groups = "drop") %>%
  mutate(Algorithm = factor(Algorithm, levels = c("Pref", "EV")))

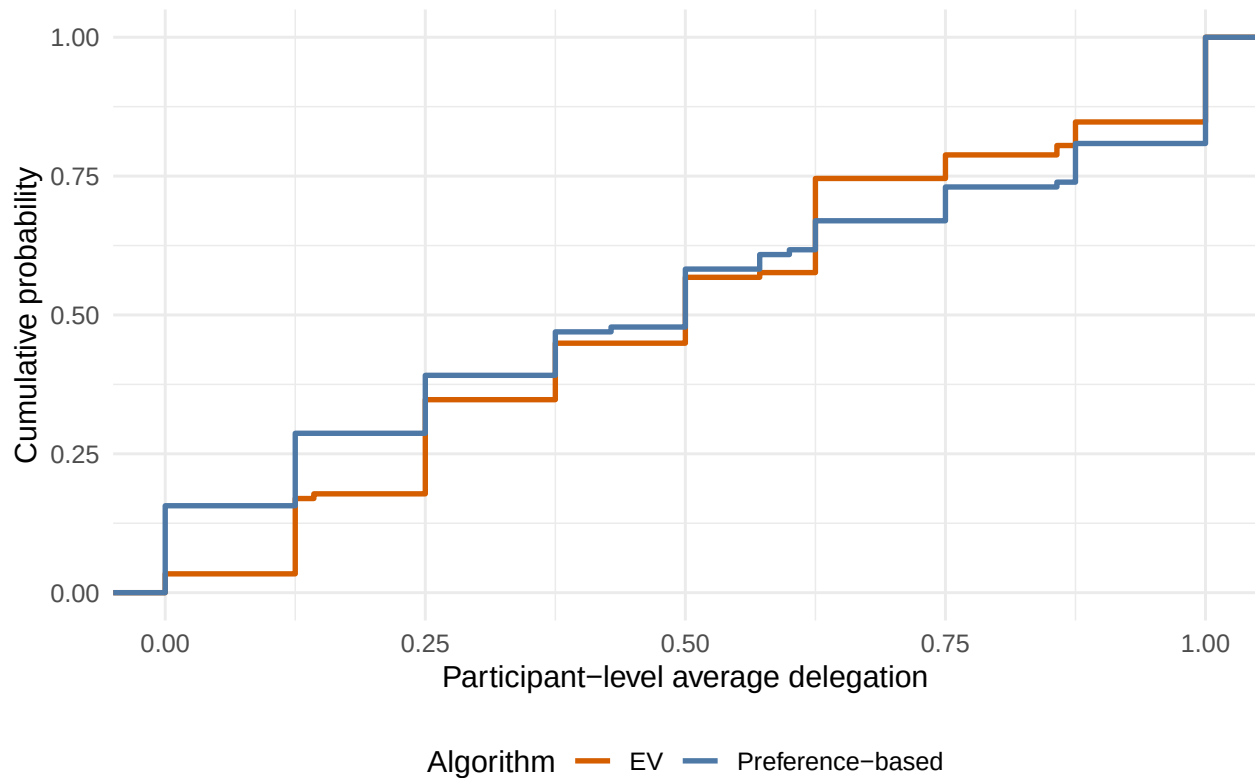
wilcox.test(avDeleg ~ Algorithm, data = delegAv, alternative = "less")

##
## Wilcoxon rank sum test with continuity correction
##
## data: avDeleg by Algorithm
## W = 6428, p-value = 0.2425
## alternative hypothesis: true location shift is less than 0
```

### 4.2 Figure 3 — ECDF of participant-level average delegation

```
fig3 <- delegAv %>%
  mutate(Algorithm = factor(Algorithm, levels = c("EV", "Pref"),
                           labels = c("EV", "Preference-based"))) %>%
  ggplot(aes(x = avDeleg, color = Algorithm)) +
  stat_ecdf(linewidth = 1) +
  scale_color_manual(values = c("EV" = col_ev, "Preference-based" = col_pref)) +
  labs(x = "Participant-level average delegation",
       y = "Cumulative probability",
       color = "Algorithm") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

fig3
```



```
ggsave("Figure3_ECDF.pdf", fig3, width = 7, height = 4.5)
```

### 4.3 Preregistered regression specification

LPM with decision-set fixed effects and participant-clustered SEs. We first construct the feedback-history regressor (running share of zero outcomes among prior delegated choices).

```
choices <- choices %>%
  arrange(ProlificID, WithinPartDecisionNumber) %>%
  group_by(ProlificID) %>%
  mutate(
    delegated_round = (Delegated == 1),
    zero_payoff      = (RelevantPayoff == 0 & Delegated == 1),
    cum_delegated    = lag(cumsum(delegated_round), default = 0),
    cum_zero         = lag(cumsum(zero_payoff), default = 0),
    zero_rate        = ifelse(cum_delegated > 0, cum_zero / cum_delegated, NA)
  ) %>%
  ungroup()

# Preregistered model: treatment x (period + feedback history)
delRegPreReg <- feols(
  Delegated ~ EVAlgorithm * (WithinPartDecisionNumber + zero_rate) | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegPreReg)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,324
```

```
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##
##           Estimate Std. Error   t value   Pr(>|t|)
## EValgorithm      -0.048717    0.083122  -0.586090 5.5845e-01
## WithinPartDecisionNumber -0.001795    0.008257  -0.217380 8.2813e-01
## zero_rate        -0.385350    0.078139  -4.931567 1.6664e-06
## EValgorithm:WithinPartDecisionNumber 0.001877    0.011504   0.163119 8.7058e-01
## EValgorithm:zero_rate    -0.049677    0.097673  -0.508609 6.1157e-01
##
## EValgorithm
## WithinPartDecisionNumber
## zero_rate          ***
## EValgorithm:WithinPartDecisionNumber
## EValgorithm:zero_rate
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.467127      Adj. R2: 0.115951
##                      Within R2: 0.107265
```

#### 4.4 Supporting regressions

Plain treatment effect (all rounds; excluding round 1; round 1 only). The round-1-only LPM reproduces the Boschloo result ( $\beta = 0.113$ , one-sided  $p = 0.041$ ).

```
# All rounds, treatment only
delRegPlainTrmt <- feols(
  Delegated ~ EValgorithm | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)

# Excluding round 1
delRegPlainTrmtNoP1 <- feols(
  Delegated ~ EValgorithm | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber > 1)
)

# Round 1 only (mirrors the Boschloo test)
delRegP1 <- feols(
  Delegated ~ EValgorithm | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber == 1)
)

screenreg(
  list(delRegPlainTrmt, delRegPlainTrmtNoP1, delRegP1),
  custom.model.names = c("All rounds", "Rounds 2-8", "Round 1 only"),
  caption = "Treatment effect on delegation (LPM, task FE, clustered SE)"
)

##
## =====
##           All rounds  Rounds 2-8  Round 1 only
## -----
```

```

## EVALgorithm          0.02      0.00      0.11
##                    (0.04)    (0.04)    (0.06)
## -----
## Num. obs.           1848      1617      231
## Num. groups: OptionsetID      8        8        8
## R^2 (full model)      0.01      0.01      0.05
## R^2 (proj model)      0.00      0.00      0.01
## Adj. R^2 (full model)  0.01      0.01      0.01
## Adj. R^2 (proj model) -0.00     -0.00      0.01
## =====
## *** p < 0.001; ** p < 0.01; * p < 0.05

```

## 5 Categorical non-delegation

We classify participants as never-, sometimes-, or always-delegators and compare never-delegation across conditions (paper: 15.7% Pref vs. 3.4% EV).

```
choices <- choices %>%
  group_by(ProlificID) %>%
  mutate(sumDelegations = sum(Delegated)) %>%
  ungroup() %>%
  left_join(
    prolificData %>% select(Participant.id, TimeInSeconds = Time.taken),
    by = c("ProlificID" = "Participant.id")
  )

analysis_data <- choices %>%
  group_by(ProlificID) %>%
  summarise(
    EVALgorithm = first(EVALgorithm),
    sumDelegations = first(sumDelegations),
    Female = first(Female),
    Age = first(Age),
    HHIncome = first(HHIncome),
    Education = first(Education),
    ProbOfBadComputerChoices = first(ProbOfBadComputerChoices),
    ProbOfAbsurdComputerChoices = first(ProbOfAbsurdComputerChoices),
    ComputerRiskTaking = first(ComputerRiskTaking),
    PBetChoices = first(PBet_count),
    TimeInSeconds = first(TimeInSeconds),
    .groups = "drop"
  ) %>%
  mutate(
    DelegationGroup = ifelse(sumDelegations == 0, "Never", "AtLeastOnce"),
    NonDelegator = 1 * (sumDelegations == 0),
    DelegatorType = factor(
      if_else(sumDelegations == 0, "NeverDelegator",
        if_else(sumDelegations == 8, "AlwaysDelegator", "SometimesDelegator")),
      levels = c("NeverDelegator", "SometimesDelegator", "AlwaysDelegator")
    )
  ) %>%
  left_join(
    subjects %>% select(ProlificID, RiskTaking, FinMarketExperience,
      FinanciallyResponsible, AIXperience),
    by = "ProlificID"
  )

# Distribution of delegator types by condition (column 2 = EV)
prop.table(table(analysis_data$DelegatorType, analysis_data$EVALgorithm), margin = 2)

##
##              0              1
##  NeverDelegator    0.15652174 0.03389831
##  SometimesDelegator 0.65217391 0.83050847
##  AlwaysDelegator   0.19130435 0.13559322
```

## 5.1 Comparisons of never-delegators vs. delegators

The following tests reproduce the in-text comparisons (age, income, education, time, gender, risk-taking, reliability, AI experience, Part-1 satisfaction).

```
# Age
t.test(Age ~ DelegationGroup, data = analysis_data)

##
## Welch Two Sample t-test
##
## data: Age by DelegationGroup
## t = 0.32876, df = 24.878, p-value = 0.7451
## alternative hypothesis: true difference in means between group AtLeastOnce and group Never is not equal to 0
## 95 percent confidence interval:
## -5.689245 7.849952
## sample estimates:
## mean in group AtLeastOnce mean in group Never
## 42.85308 41.77273

# Time in experiment
t.test(TimeInSeconds ~ NonDelegator, data = analysis_data)

##
## Welch Two Sample t-test
##
## data: TimeInSeconds by NonDelegator
## t = -0.9713, df = 25.374, p-value = 0.3406
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
## -385.8898 138.4297
## sample estimates:
## mean in group 0 mean in group 1
## 1113.043 1236.773

# Household income (categorical)
chisq.test(table(analysis_data$DelegationGroup, analysis_data$HHIncome))

##
## Pearson's Chi-squared test
##
## data: table(analysis_data$DelegationGroup, analysis_data$HHIncome)
## X-squared = 13.171, df = 11, p-value = 0.2823

# Education (categorical)
chisq.test(table(analysis_data$DelegationGroup, analysis_data$Education))

##
## Pearson's Chi-squared test
##
## data: table(analysis_data$DelegationGroup, analysis_data$Education)
## X-squared = 1.859, df = 5, p-value = 0.8683

# Gender: never-delegation rate among males vs. females (Boschloo)
female_table <- table(
  DelegationGroup = analysis_data$DelegationGroup,
  Female = analysis_data$Female
```

```

)
exact.test(female_table[, c(2, 4)], method = "Boschloo", to.plot = FALSE)$p.value

## [1] 0.1561955

# Self-reported financial risk-taking, reliability, AI experience
t.test(RiskTaking ~ NonDelegator, data = analysis_data)

##
## Welch Two Sample t-test
##
## data: RiskTaking by NonDelegator
## t = -0.22379, df = 24.99, p-value = 0.8247
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
## -1.540832 1.238807
## sample estimates:
## mean in group 0 mean in group 1
## 4.530806 4.681818

t.test(FinanciallyResponsible ~ NonDelegator, data = analysis_data)

##
## Welch Two Sample t-test
##
## data: FinanciallyResponsible by NonDelegator
## t = 1.229, df = 24.512, p-value = 0.2307
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
## -0.6803629 2.6889799
## sample estimates:
## mean in group 0 mean in group 1
## 6.549763 5.545455

t.test(AIXperience ~ NonDelegator, data = analysis_data)

##
## Welch Two Sample t-test
##
## data: AIXperience by NonDelegator
## t = 0.86582, df = 24.274, p-value = 0.3951
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
## -0.9475606 2.3185214
## sample estimates:
## mean in group 0 mean in group 1
## 5.867299 5.181818

# Within the preference-based condition: Part-1 satisfaction by delegation type
NeverDelegate <- choices %>%
  select(ProlificID, sumDelegations) %>%
  distinct() %>%
  mutate(sumDeleg0 = (sumDelegations == 0))

subjects2 <- subjects %>% left_join(NeverDelegate, by = "ProlificID")

```

```

subjects2 %>%
  filter(EVAlgorithm == 0) %>%
  group_by(sumDeleg0) %>%
  summarise(SatisfiedPart1 = mean(SatisfiedPart1, na.rm = TRUE),
            n = n(), .groups = "drop")

```

```

## # A tibble: 2 x 3
##   sumDeleg0 SatisfiedPart1      n
##   <lgl>          <dbl> <int>
## 1 FALSE          7.56     97
## 2 TRUE           7.94     18

```

## 6 Responses to feedback (win-stay / lose-shift)

Win-stay/lose-shift model on rounds 2–8 (paper, Section “Responses to feedback”).

```
choices <- choices %>%
  group_by(ProlificID) %>%
  arrange(WithinPartDecisionNumber, .by_group = TRUE) %>%
  mutate(
    lagDelegated = lag(Delegated, 1),
    lagRelevantPayoff = lag(RelevantPayoff, 1),
    lagSatisfactionRating = lag(SatisfactionRating, 1)
  ) %>%
  ungroup()

choices$stay <- 1 * (choices$Delegated == choices$lagDelegated)

# "Prior win": the outcome of the PREVIOUS round was strictly positive.
# (lagWin is defined from the lagged payoff, i.e. the genuine win-stay/lose-shift
# notion of reacting to the previously experienced outcome.)
choices$lagWin <- 1 * (choices$lagRelevantPayoff > 0)

# Pooled model: treatment x prior win
winStayReg <- feols(
  stay ~ EVALgorithm * lagWin | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg)

## OLS estimation, Dep. Var.: stay
## Observations: 1,615
## Fixed-effects: OptionsetID: 8, WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##               Estimate Std. Error   t value   Pr(>|t|)
## EVALgorithm    -0.104670   0.049740 -2.104353 3.6424e-02 *
## lagWin          0.169470   0.035178  4.817493 2.6258e-06 ***
## EVALgorithm:lagWin 0.008762   0.048728  0.179822 8.5745e-01
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.456921   Adj. R2: 0.052558
##               Within R2: 0.043757
```

Separately by treatment, distinguishing whether the previous round was delegated (`lagDelegated == 1`, i.e. “the algorithm” produced the outcome) from a self-made choice. This separates lose-shift after one’s own loss from lose-shift after a delegated (algorithmic) loss.

```
# EV condition
winStayReg_EV <- feols(
  stay ~ lagDelegated * I(1 - lagWin) | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, EVALgorithm == 1 & Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg_EV)
```

```
## OLS estimation, Dep. Var.: stay
```

```

## Observations: 819
## Fixed-effects: OptionsetID: 8, WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##               Estimate Std. Error   t value Pr(>|t|)
## lagDelegated      0.044647   0.044771   0.997214 0.3207185
## I(1 - lagWin)     -0.075637   0.050734 -1.490836 0.1386957
## lagDelegated:I(1 - lagWin) -0.184727   0.069840 -2.645020 0.0092905 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.473051      Adj. R2: 0.03568
##               Within R2: 0.038964

# Preference-based condition
winStayReg_Pref <- feols(
  stay ~ lagDelegated * I(1 - lagWin) | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, EVALgorithm == 0 & Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg_Pref)

## OLS estimation, Dep. Var.: stay
## Observations: 796
## Fixed-effects: OptionsetID: 8, WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##               Estimate Std. Error   t value Pr(>|t|)
## lagDelegated      0.024197   0.036355   0.665574 0.507028
## I(1 - lagWin)     -0.108747   0.048066 -2.262433 0.025565 *
## lagDelegated:I(1 - lagWin) -0.140381   0.063708 -2.203504 0.029570 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.432604      Adj. R2: 0.058073
##               Within R2: 0.042744

# Average stay rates by condition, prior delegation, and prior win/loss
choices %>%
  filter(Part == 2, WithinPartDecisionNumber > 1) %>%
  group_by(EVALgorithm, lagDelegated, lagWin) %>%
  summarise(avg_stay_rate = mean(stay, na.rm = TRUE),
            n = n(), .groups = "drop") %>%
  arrange(EVALgorithm, lagDelegated, lagWin)

## # A tibble: 9 x 5
##   EVALgorithm lagDelegated lagWin avg_stay_rate    n
##   <int>         <int>   <dbl>         <dbl> <int>
## 1         0           0     0           0.677   155
## 2         0           0     1           0.769   255
## 3         0           1     0           0.547   139
## 4         0           1     1           0.785   247
## 5         0          NA    NA           NaN     2
## 6         1           0     0           0.583   144
## 7         1           0     1           0.668   262
## 8         1           1     0           0.444   169
## 9         1           1     1           0.697   244

```

## 7 Exploratory mechanisms

### 7.1 Coefficient model (interaction with performance expectations)

LPM of individual delegation decisions interacting treatment with participants' algorithmic performance expectations, controlling for round, per-choice Part-1 confidence, revealed Part-1 risk preferences, and task fixed effects.

```
delRegWConf <- feols(  
  Delegated ~ EValgorithm * (WithinPartDecisionNumber + ConfidenceP1 +  
    ProbOfAbsurdComputerChoices + ProbOfBadComputerChoices +  
    EVChoices + truePChoices) | OptionsetID,  
  cluster = ~ProlificID,  
  data = subset(choices, Part == 2)  
)  
summary(delRegWConf)
```

```
## OLS estimation, Dep. Var.: Delegated  
## Observations: 1,848  
## Fixed-effects: OptionsetID: 8  
## Standard-errors: Clustered (ProlificID)  
##  
##               Estimate Std. Error   t value  
## EValgorithm          0.856469   0.891957   0.960213  
## WithinPartDecisionNumber -0.009920   0.005711  -1.737089  
## ConfidenceP1          -0.025275   0.012620  -2.002676  
## ProbOfAbsurdComputerChoices  0.009864   0.018425   0.535348  
## ProbOfBadComputerChoices  -0.060048   0.016324  -3.678547  
## EVChoices             0.067083   0.058904   1.138847  
## truePChoices           0.058960   0.054107   1.089695  
## EValgorithm:WithinPartDecisionNumber -0.012262   0.008341  -1.470058  
## EValgorithm:ConfidenceP1      0.043682   0.015740   2.775229  
## EValgorithm:ProbOfAbsurdComputerChoices -0.072908   0.023059  -3.161780  
## EValgorithm:ProbOfBadComputerChoices  0.036186   0.020603   1.756318  
## EValgorithm:EVChoices        -0.089742   0.077330  -1.160496  
## EValgorithm:truePChoices     -0.095940   0.071561  -1.340675  
##  
##               Pr(>|t|)  
## EValgorithm          0.3379475  
## WithinPartDecisionNumber  0.0836987 .  
## ConfidenceP1          0.0463759 *  
## ProbOfAbsurdComputerChoices  0.5929221  
## ProbOfBadComputerChoices  0.0002916 ***  
## EVChoices             0.2559415  
## truePChoices           0.2769783  
## EValgorithm:WithinPartDecisionNumber  0.1429009  
## EValgorithm:ConfidenceP1      0.0059660 **  
## EValgorithm:ProbOfAbsurdComputerChoices 0.0017772 **  
## EValgorithm:ProbOfBadComputerChoices  0.0803535 .  
## EValgorithm:EVChoices        0.2470398  
## EValgorithm:truePChoices     0.1813374  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
## RMSE: 0.474486      Adj. R2: 0.089466  
##                   Within R2: 0.089764
```

## 7.2 Figure A.1 — Condition-specific marginal effects

```

coefs_wc <- coef(delRegWConf)
vcm_wc   <- vcov(delRegWConf)

# SE of the sum of two coefficients (main effect + interaction)
se_sum <- function(v1, v2) {
  sqrt(vcm_wc[v1, v1] + vcm_wc[v2, v2] + 2 * vcm_wc[v1, v2])
}

coef_spec <- data.frame(
  var_name = c("WithinPartDecisionNumber", "ConfidenceP1",
               "ProbOfBadComputerChoices", "ProbOfAbsurdComputerChoices",
               "EVChoices", "truePChoices"),
  var_label = c("Decision round", "Confidence (Part 1)",
                "Number of non-preferred choices",
                "Number of completely weird choices",
                "EV-maximising choices (Part 1)",
                "P-bet choices when not EV-maximising (Part 1)"),
  stringsAsFactors = FALSE
)

plot_data <- data.frame()
for (i in seq_len(nrow(coef_spec))) {
  var_name <- coef_spec$var_name[i]
  var_label <- coef_spec$var_label[i]

  pb_est <- coefs_wc[var_name]
  pb_se <- sqrt(vcm_wc[var_name, var_name])

  int_name <- paste0("EVALgorithm:", var_name)
  if (int_name %in% names(coefs_wc)) {
    ev_est <- pb_est + coefs_wc[int_name]
    ev_se <- se_sum(var_name, int_name)
  } else {
    ev_est <- pb_est
    ev_se <- pb_se
  }

  plot_data <- rbind(plot_data, data.frame(
    Variable = var_label,
    Condition = c("Preference-based", "EV"),
    Estimate = c(pb_est, ev_est),
    SE = c(pb_se, ev_se),
    stringsAsFactors = FALSE
  ))
}

plot_data <- plot_data %>%
  mutate(
    Variable = factor(Variable, levels = coef_spec$var_label),
    Condition = factor(Condition, levels = c("Preference-based", "EV")),
    CI_lo = Estimate - 1.96 * SE,
    CI_hi = Estimate + 1.96 * SE,

```

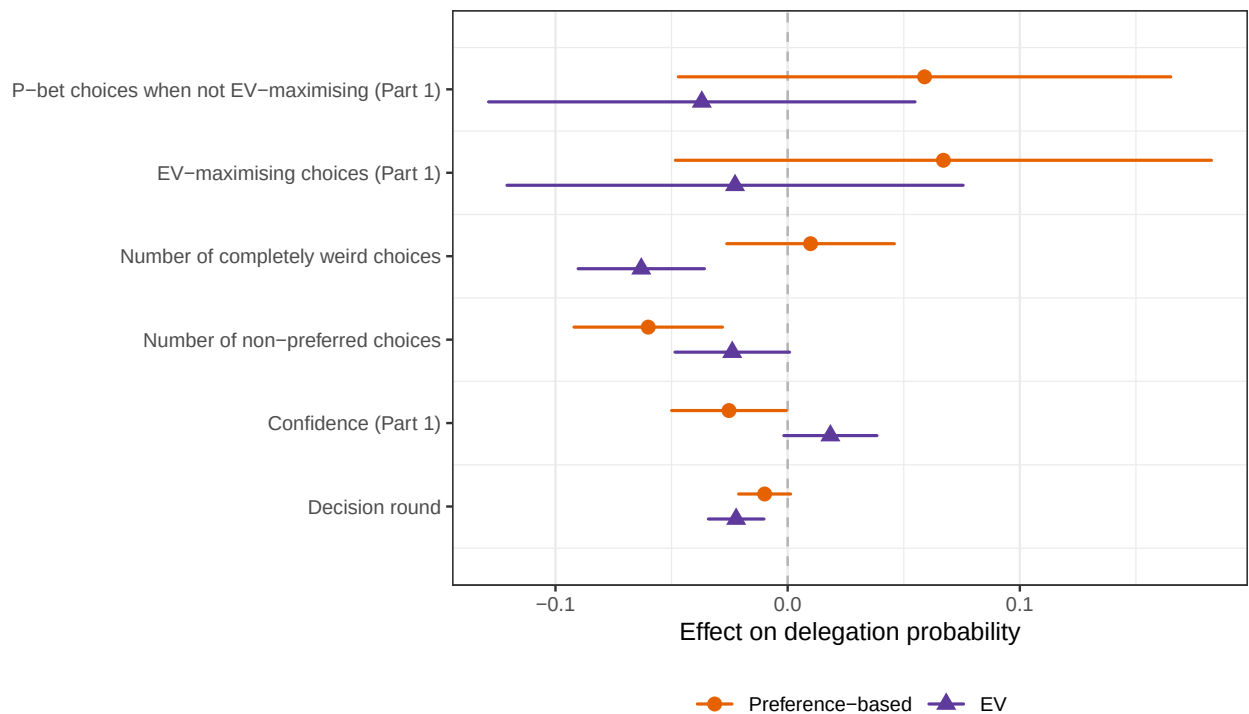
```

    y_pos = as.numeric(Variable) +
      ifelse(Condition == "Preference-based", 0.15, -0.15)
  )

figA1 <- ggplot(plot_data) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray70") +
  geom_linerange(aes(xmin = CI_lo, xmax = CI_hi, y = y_pos, color = Condition),
    linewidth = 0.6, lineend = "round") +
  geom_point(aes(x = Estimate, y = y_pos, color = Condition, shape = Condition),
    size = 2.5) +
  scale_y_continuous(breaks = seq_len(nrow(coef_spec)),
    labels = levels(plot_data$Variable),
    expand = c(0.15, 0)) +
  scale_color_manual(values = c("Preference-based" = "#E66100", "EV" = "#5D3A9B")) +
  scale_shape_manual(values = c("Preference-based" = 16, "EV" = 17)) +
  labs(x = "Effect on delegation probability", y = NULL,
    color = NULL, shape = NULL,
    caption = paste0("95% confidence intervals. Linear probability model with ",
      "task fixed effects and participant-clustered SEs.\n",
      "EV-condition effects are main effect + interaction term.))" +
  theme_bw(base_size = 10) +
  theme(legend.position = "bottom",
    panel.grid.major.y = element_blank(),
    plot.caption = element_text(hjust = 0, size = 9))

```

figA1



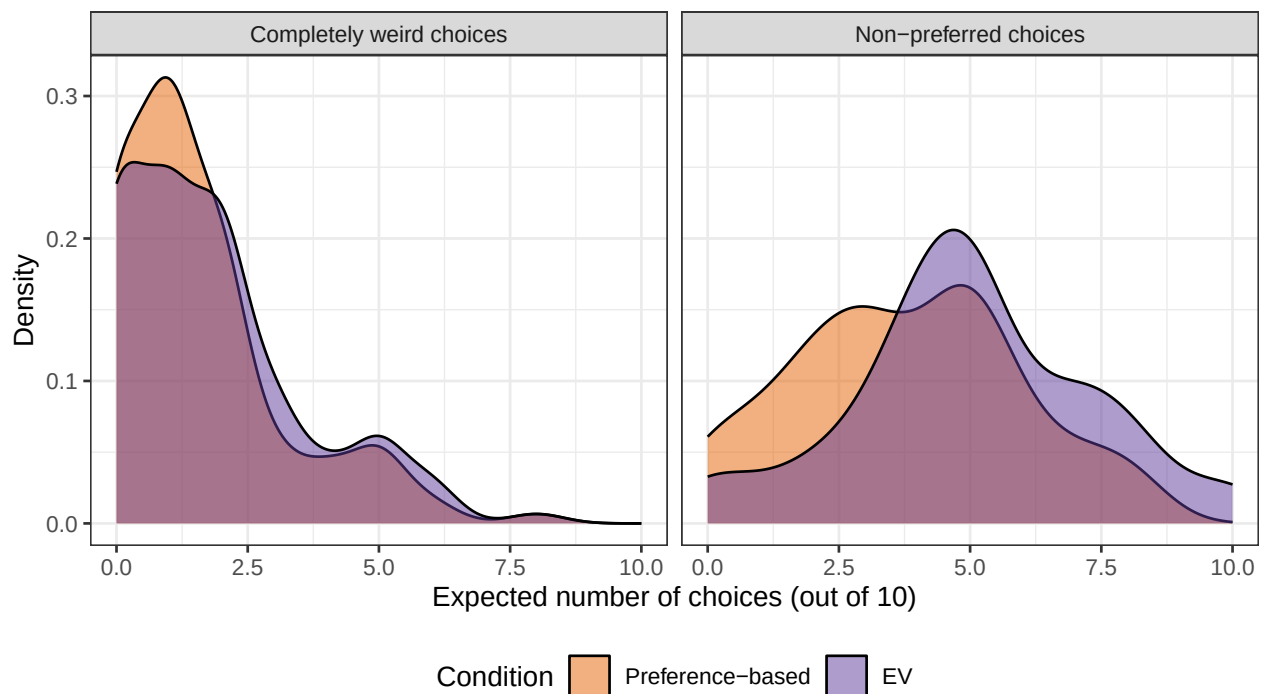
95% confidence intervals. Linear probability model with task fixed effects and part EV-condition effects are main effect + interaction term.

```
ggsave("CoefPlotBadAbsurd.pdf", figA1, width = 7, height = 4.5)
```

### 7.3 Figure A.2 — Expected non-preferred and completely weird choices

```
figA2 <- subjects %>%
  pivot_longer(cols = c(ProbOfBadComputerChoices, ProbOfAbsurdComputerChoices),
    names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = value, fill = factor(EVALgorithm))) +
  geom_density(alpha = 0.5) +
  facet_wrap(~ variable, labeller = labeller(variable = c(
    ProbOfBadComputerChoices = "Non-preferred choices",
    ProbOfAbsurdComputerChoices = "Completely weird choices"
  ))) +
  scale_fill_manual(values = c("0" = "#E66100", "1" = "#5D3A9B"),
    labels = c("0" = "Preference-based", "1" = "EV"),
    name = "Condition") +
  labs(x = "Expected number of choices (out of 10)", y = "Density") +
  theme_bw() +
  theme(legend.position = "bottom")
```

figA2



```
ggsave("ExpectedBadAndAbsurdChoicesByTrmt.pdf", figA2, width = 7, height = 4)
```

### 7.4 Treatment comparison of expected algorithmic errors

Formal comparison of expected non-preferred and completely weird choices across treatments. A LOWER expected number of non-preferred choices under the preference-based algorithm would indicate that participants believe the personalized algorithm is doing its job (matching their preferences).

```
expect_tbl <- subjects %>%
  mutate(Condition = ifelse(EVAlgorithm == 1, "EV", "Preference-based")) %>%
  group_by(Condition) %>%
  summarise(
    n = n(),
    mean_nonpreferred = mean(ProbOfBadComputerChoices, na.rm = TRUE),
    sd_nonpreferred = sd(ProbOfBadComputerChoices, na.rm = TRUE),
    mean_weird = mean(ProbOfAbsurdComputerChoices, na.rm = TRUE),
    sd_weird = sd(ProbOfAbsurdComputerChoices, na.rm = TRUE),
    .groups = "drop"
  )
expect_tbl
```

```
## # A tibble: 2 x 6
##   Condition      n mean_nonpreferred sd_nonpreferred mean_weird sd_weird
##   <chr>      <int>          <dbl>          <dbl>          <dbl>    <dbl>
## 1 EV          118              5              2.32          1.78     1.74
## 2 Preference-based 115              3.78              2.16          1.57     1.62
```

```
# Expected NON-PREFERRED choices: EV vs. Preference-based
t.test(ProbOfBadComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
## Welch Two Sample t-test
##
## data: ProbOfBadComputerChoices by EVAlgorithm
## t = -4.1504, df = 230.54, p-value = 4.671e-05
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
## -1.7953130 -0.6394696
## sample estimates:
## mean in group 0 mean in group 1
## 3.782609 5.000000
```

```
wilcox.test(ProbOfBadComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
## Wilcoxon rank sum test with continuity correction
##
## data: ProbOfBadComputerChoices by EVAlgorithm
## W = 4805, p-value = 9.898e-05
## alternative hypothesis: true location shift is not equal to 0
```

```
# Expected COMPLETELY WEIRD choices: EV vs. Preference-based
t.test(ProbOfAbsurdComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
## Welch Two Sample t-test
##
## data: ProbOfAbsurdComputerChoices by EVAlgorithm
## t = -0.93521, df = 230.48, p-value = 0.3507
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
## -0.6392193 0.2277233
## sample estimates:
```

```
## mean in group 0 mean in group 1
##      1.573913      1.779661

wilcox.test(ProbOfAbsurdComputerChoices ~ EVALgorithm, data = subjects)

##
## Wilcoxon rank sum test with continuity correction
##
## data: ProbOfAbsurdComputerChoices by EVALgorithm
## W = 6328, p-value = 0.361
## alternative hypothesis: true location shift is not equal to 0
```

## 8 Post-task reasons

### 8.1 Figure 4 — Reasons by treatment and delegation type

Reason-selection frequencies for the hypothetical additional-task choice, by treatment (EV vs. preference-based) and delegation type (delegators vs. never-delegators).

```
subjects2 <- subjects %>% left_join(NeverDelegate, by = "ProlificID")

subjects_long <- subjects2 %>%
  pivot_longer(cols = DontTrust:Other, names_to = "item", values_to = "clicked")

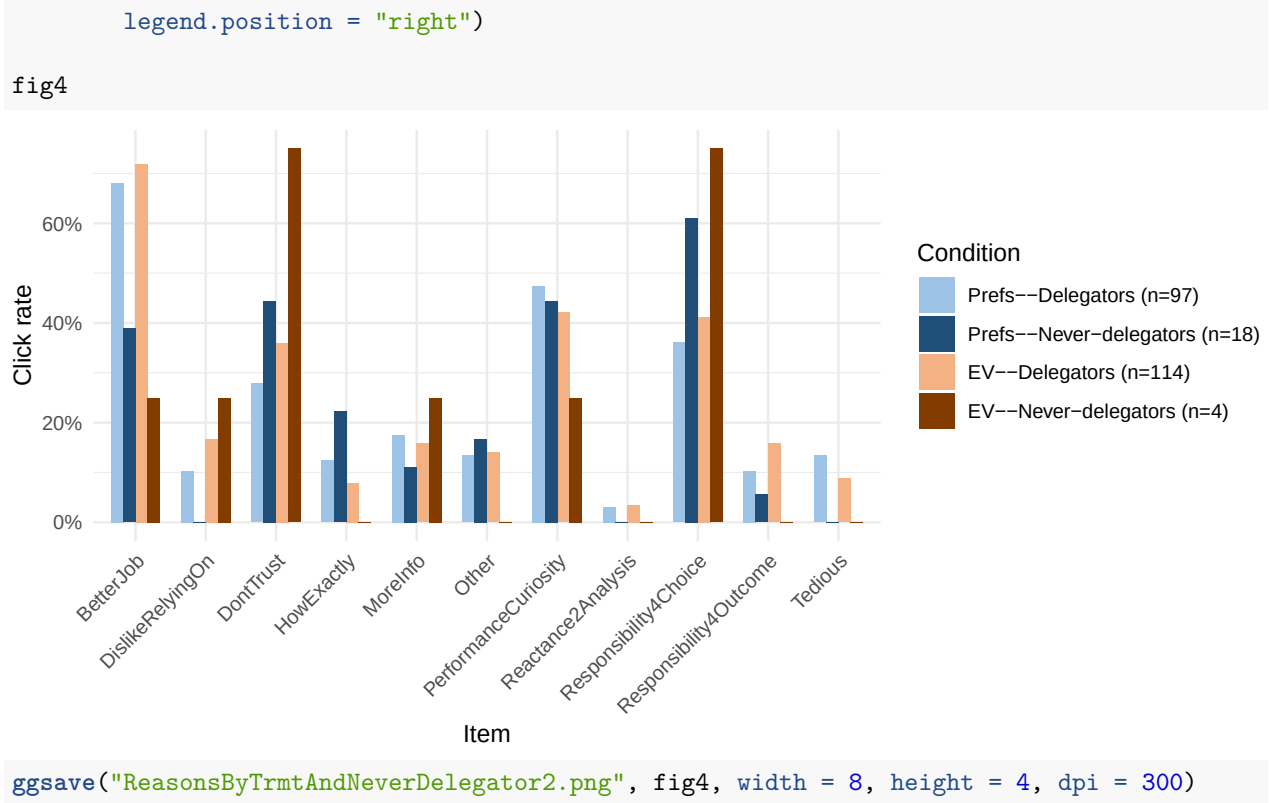
# Group sizes for legend labels
n_labels <- subjects_long %>%
  group_by(sumDeleg0, EVALgorithm) %>%
  summarise(n = n_distinct(ProlificID), .groups = "drop") %>%
  mutate(
    group = case_when(
      sumDeleg0 == TRUE & EVALgorithm == 0 ~ "Prefs--Never-delegators",
      sumDeleg0 == TRUE & EVALgorithm == 1 ~ "EV--Never-delegators",
      sumDeleg0 == FALSE & EVALgorithm == 0 ~ "Prefs--Delegators",
      sumDeleg0 == FALSE & EVALgorithm == 1 ~ "EV--Delegators"
    ),
    group_label = paste0(group, " (n=", n, ")")
  )

group_colors <- c(
  "Prefs--Delegators" = "#9dc3e6",
  "Prefs--Never-delegators" = "#1f4e79",
  "EV--Delegators" = "#f4b183",
  "EV--Never-delegators" = "#833c00"
)

label_map <- setNames(n_labels$group_label, n_labels$group)

click_rates_both <- subjects_long %>%
  group_by(sumDeleg0, EVALgorithm, item) %>%
  summarise(rate = mean(clicked, na.rm = TRUE), .groups = "drop") %>%
  mutate(
    group = case_when(
      sumDeleg0 == TRUE & EVALgorithm == 0 ~ "Prefs--Never-delegators",
      sumDeleg0 == TRUE & EVALgorithm == 1 ~ "EV--Never-delegators",
      sumDeleg0 == FALSE & EVALgorithm == 0 ~ "Prefs--Delegators",
      sumDeleg0 == FALSE & EVALgorithm == 1 ~ "EV--Delegators"
    ),
    group = factor(group, levels = names(group_colors))
  )

fig4 <- ggplot(click_rates_both, aes(x = item, y = rate, fill = group)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.7) +
  scale_fill_manual(values = group_colors, labels = label_map,
    breaks = names(group_colors), name = "Condition") +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(x = "Item", y = "Click rate") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
```



## 8.2 Aggregate reason patterns

In-text figures for performance, distrust, and responsibility motives by delegation type (delegators vs. never-delegators).

```
subjects_long %>%
  group_by(sumDeleg0, item) %>%
  summarise(rate = mean(clicked, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = sumDeleg0, values_from = rate) %>%
  rename(Delegators = `FALSE`, NeverDelegators = `TRUE`)
```

```
## # A tibble: 11 x 3
##   item                Delegators NeverDelegators
##   <chr>              <dbl>         <dbl>
## 1 BetterJob          0.701           0.364
## 2 DislikeRelyingOn    0.137           0.0455
## 3 DontTrust           0.322           0.5
## 4 HowExactly          0.0995          0.182
## 5 MoreInfo           0.166           0.136
## 6 Other              0.137           0.136
## 7 PerformanceCuriosity 0.445           0.409
## 8 Reactance2Analysis  0.0332          0
## 9 Responsibility4Choice 0.389           0.636
## 10 Responsibility4Outcome 0.133           0.0455
## 11 Tedious           0.109           0
```

### 8.3 LPM: never-delegation on reasons

```
reason_vars <- c("DontTrust", "BetterJob", "HowExactly", "MoreInfo",
  "Responsibility4Choice", "Responsibility4Outcome",
  "DislikeRelyingOn", "Reactance2Analysis",
  "PerformanceCuriosity", "Tedious", "Other")

# Bivariate LPMs (one per reason, HC2 SEs)
bivariate_tbl <- do.call(rbind, lapply(reason_vars, function(rv) {
  m <- lm(as.formula(paste("as.numeric(sumDeleg0) ~", rv)), data = subjects2)
  ct <- coeftest(m, vcov = vcovHC(m, type = "HC2"))
  data.frame(reason = rv,
    coef = round(ct[2, 1], 3), se = round(ct[2, 2], 3),
    t = round(ct[2, 3], 3), p = round(ct[2, 4], 3))
}))
print(bivariate_tbl, row.names = FALSE)
```

```
##           reason   coef   se    t    p
##      DontTrust  0.068 0.044  1.528 0.128
##      BetterJob -0.131 0.048 -2.739 0.007
##      HowExactly  0.073 0.077  0.950 0.343
##      MoreInfo  -0.018 0.049 -0.376 0.707
## Responsibility4Choice  0.087 0.041  2.111 0.036
## Responsibility4Outcome -0.068 0.041 -1.688 0.093
##      DislikeRelyingOn -0.070 0.040 -1.769 0.078
##      Reactance2Analysis -0.097 0.020 -4.926 0.000
##      PerformanceCuriosity -0.013 0.038 -0.328 0.743
##              Tedious  -0.105 0.021 -4.945 0.000
##              Other   -0.001 0.056 -0.014 0.989
```

```
# Joint LPM: full sample + preference-treatment only
f_joint <- as.formula(paste("as.numeric(sumDeleg0) ~",
  paste(reason_vars, collapse = " + ")))
lpm_all <- lm(f_joint, data = subjects2)
lpm_pref <- lm(f_joint, data = subjects2[subjects2$EVALgorithm == 0, ])

vcov_all <- vcovHC(lpm_all, type = "HC2")
vcov_pref <- vcovHC(lpm_pref, type = "HC2")

screenreg(
  list(lpm_all, lpm_pref),
  override.se = list(sqrt(diag(vcov_all)), sqrt(diag(vcov_pref))),
  override.pvalues = list(
    2 * pt(abs(coef(lpm_all)) / sqrt(diag(vcov_all))),
    df = lpm_all$df.residual, lower.tail = FALSE),
    2 * pt(abs(coef(lpm_pref)) / sqrt(diag(vcov_pref))),
    df = lpm_pref$df.residual, lower.tail = FALSE)
),
custom.model.names = c("Full sample", "Pref. treatment"),
caption = "LPM: never-delegation ~ reasons (HC2 SEs)"
)
```

```
##
## =====
```

```

##                               Full sample  Pref. treatment
## -----
## (Intercept)                   0.14 **      0.20 *
##                               (0.05)      (0.09)
## DontTrust                     0.08         0.15
##                               (0.04)      (0.09)
## BetterJob                    -0.11 *       -0.14
##                               (0.05)      (0.08)
## HowExactly                   0.04         0.05
##                               (0.08)      (0.10)
## MoreInfo                    -0.04         -0.15
##                               (0.06)      (0.11)
## Responsibility4Choice         0.06         0.11
##                               (0.04)      (0.07)
## Responsibility4Outcome       -0.04         -0.00
##                               (0.04)      (0.10)
## DislikeRelyingOn            -0.09 *       -0.25 **
##                               (0.04)      (0.08)
## Reactance2Analysis          -0.09         -0.12
##                               (0.04)      (0.12)
## PerformanceCuriosity        0.02         0.03
##                               (0.04)      (0.07)
## Tedious                     -0.07 **      -0.16 **
##                               (0.03)      (0.06)
## Other                       -0.02         -0.01
##                               (0.06)      (0.12)
## -----
## R^2                          0.10         0.16
## Adj. R^2                     0.05         0.06
## Num. obs.                    233         115
## =====
## *** p < 0.001; ** p < 0.01; * p < 0.05

```

## 9 Session information

```
sessionInfo()
```

```

## R version 4.6.1 (2026-06-24)
## Platform: x86_64-pc-linux-gnu
## Running under: TUXEDO OS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=de_CH.UTF-8      LC_NUMERIC=C
##  [3] LC_TIME=de_CH.UTF-8      LC_COLLATE=de_CH.UTF-8
##  [5] LC_MONETARY=de_CH.UTF-8  LC_MESSAGES=de_CH.UTF-8
##  [7] LC_PAPER=de_CH.UTF-8     LC_NAME=C
##  [9] LC_ADDRESS=C             LC_TELEPHONE=C
## [11] LC_MEASUREMENT=de_CH.UTF-8 LC_IDENTIFICATION=C
##

```

```

## time zone: Europe/Zurich
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] texreg_1.39.5  sandwich_3.1-1  lmtest_0.9-40  zoo_1.8-15     fixest_0.14.1
## [6] Exact_3.3      scales_1.4.0    ggplot2_4.0.3  tidyr_1.3.2    dplyr_1.2.1
##
## loaded via a namespace (and not attached):
## [1] utf8_1.2.6      generics_0.1.4    dremarr_1.5.0
## [4] lattice_0.22-9  digest_0.6.39     magrittr_2.0.5
## [7] evaluate_1.0.5  grid_4.6.1        RColorBrewer_1.1-3
## [10] fastmap_1.2.0   Formula_1.2-5     tinytex_0.59
## [13] httr_1.4.8      purrr_1.2.2       stringmagic_1.2.0
## [16] numDeriv_2016.8-1.1 textshaping_1.0.5 cli_3.6.6
## [19] rlang_1.2.0     withr_3.0.2       yaml_2.3.12
## [22] rootSolve_1.8.2.4 tools_4.6.1       vctrs_0.7.3
## [25] R6_2.6.1        lifecycle_1.0.5   ragg_1.5.2
## [28] pkgconfig_2.0.3 pillar_1.11.1     gtable_0.3.6
## [31] glue_1.8.1      Rcpp_1.1.1-1.1    systemfonts_1.3.2
## [34] xfun_0.57       tibble_3.3.1      tidysselect_1.2.1
## [37] rstudioapi_0.18.0 knitr_1.51        farver_2.1.2
## [40] htmltools_0.5.9 nlme_3.1-169      rmarkdown_2.31
## [43] labeling_0.4.3   compiler_4.6.1    S7_0.2.2

```